

Case Study Solution

Catching a Drone with a Robot Arm
Using a Gripper-Mounted HD Camera

Deepanshu Makkar

February 16, 2026

Contents

1. Problem Setup and Assumptions	2
2. Algorithm Overview: What kind of algorithm could be used?	2
3. Challenges of Using a Single Camera (and how to resolve them)	5
4. Alternative Approaches and Additional Sensors	5

1. Problem Setup and Assumptions

1.1 Problem Setup

A robot arm equipped with a gripper and an integrated full HD camera near the gripper should detect, track, and capture a drone in its workspace. The goal is to estimate the drone pose robustly enough to guide the robot end-effector into a feasible capture configuration and close the gripper safely.

1.2 Assumptions

- The integrated camera is a standard frame-based camera (RGB or grayscale) with a fixed frame rate and known field-of-view.
- The camera is calibrated (intrinsics available) and mounted rigidly to the gripper/end-effector (hand-eye calibration available or can be performed).
- Robot forward kinematics are available to map the current gripper pose into the robot base coordinate system.
- The approximate physical size of the drone is known (or can be constrained), enabling practical monocular depth estimation strategies.
- The drone remains within the robot workspace long enough for interception and is not permanently occluded by the environment.
- Deploy on a Embedded Device like Nvidia Jetson Nano

2. Algorithm Overview: What kind of algorithm could be used?

2.1 Summary

I would propose combining motion-guided drone detection with probabilistic tracking and latency-aware interception control. A YOLOMG-style detector can fuse an ego-motion compensated difference map with RGB to detect extremely small drones in complex backgrounds, after which 3D target position can be inferred from monocular cues and transformed into the robot base frame. A Bayesian filter can provide smooth position and velocity estimates and enable short-horizon prediction for delay compensation. Finally, the robot can align the gripper to a defined capture point and close only when safety and confidence conditions are satisfied, with verification and recovery logic for failed grasps.

2.2 High-Level Pipeline

1. Detect the drone in each camera frame.

Since the target drone can be extremely small (sometimes only a few pixels) and easily confused with complex backgrounds, I would propose employing a motion-guided detector such as *YOLOMG* [1]. As a strong real-time baseline (especially when motion cues are less reliable, e.g., slow/hovering targets), a modern single-stage detector such as YOLOv8 can also be considered [3]. Both options provide per-frame bounding boxes and confidence scores as the primary measurements.

2. Generate consistent measurements from detections.

Per-frame detections are treated as noisy observations. A short-term association stage rejects spurious detections and maintains a consistent target hypothesis across frames; in practice, a tracking-by-detection association method such as ByteTrack is a robust choice to handle score fluctuations and occasional missed detections while keeping the pipeline lightweight [2]. This produces temporally consistent measurements for the downstream state estimator.

3. Estimate 3D position from monocular observations.

The image detection is converted into a viewing ray using camera calibration. Since a single camera does not directly provide depth, range is inferred via practical constraints such as approximate known drone size, keypoint-based pose estimation, and/or controlled camera motion to induce parallax.

4. Transform measurements into the robot base frame.

Using the rigid camera-to-gripper mounting and robot kinematics, the estimated drone position is expressed in the robot base coordinate system to support interception planning and grasp execution.

5. Estimate the drone state using a Bayesian filter.

A state-space estimator fuses measurements over time to produce smooth position and velocity estimates, robust to noise, jitter, and occasional dropouts. The velocity estimate is used for interception and delay compensation.

6. Predict the future target state to compensate latency.

The estimated state is propagated forward over a short horizon to account for sensing, computation, and actuation delays, improving interception performance for fast targets.

7. Control the robot using interception or visual servoing.

A desired capture pose is defined relative to the gripper. The robot is driven toward the predicted target while respecting speed/acceleration and workspace constraints, maintaining target visibility.

8. Execute capture with safety gating and verification.

The gripper closes only when proximity, relative velocity, and confidence thresholds are satisfied, followed by capture verification and safe retract; otherwise the system retries conservatively.

2.3 Pseudocode (high-level)

Inputs:

- Camera stream (RGB frames)
- Camera calibration (intrinsics)
- Camera-to-gripper mounting calibration
- Robot kinematics and controller
- Approx. drone size (optional) / depth strategy settings
- Safety thresholds (distance, relative speed, confidence)

State:

- Track manager (association across frames, e.g., ByteTrack-style)
- Bayesian filter state (position + velocity estimate)
- Capture mode flag (approach / capture / retract)

Loop (each new frame):

1) Acquire frame

`frame = get_camera_frame()`

2) Detect drone (choose one)

`dets = YOLOMG(frame, prev_frames)` OR `dets = YOLOv8(frame)`

each det has: bbox, confidence

3) Association / measurement cleaning

`track = associate_and_update_tracks(dets)`

if no valid track:

keep predicting filter briefly (short timeout)

```

continue

meas_2d = track.bbox_center
meas_conf = track.confidence

4) Monocular 3D estimation (camera coordinates)
bearing_ray = backproject_to_ray(meas_2d, camera_intrinsics)

depth = estimate_depth(
    bearing_ray,
    using_depth_from_size / keypoints / controlled_parallax
)

if depth not reliable:
    continue # or reduce speed / re-acquire

meas_3d_cam = ray_to_3d(bearing_ray, depth)

5) Transform to robot base frame
meas_3d_base = transform_cam_to_base(
    meas_3d_cam,
    camera_to_gripper_calib,
    robot_forward_kinematics()
)

6) Bayesian filtering (state estimation)
filter.predict(dt)
filter.update(meas_3d_base, meas_conf)

7) Latency compensation
pred_target = filter.predict_forward(short_horizon)

8) Robot interception control
desired_capture_pose = compute_capture_pose(pred_target)
command_robot_toward(desired_capture_pose, safety_limits)

9) Safety gating + capture
if close_enough(pred_target) AND
relative_speed_low(filter.velocity) AND
confidence_stable(track):
    close_gripper()

if verify_grasp(force_feedback OR visual_check):
    retract_to_safe_pose()
    break / return SUCCESS
else:
    open_gripper()
    reinitialize_track_and_filter()
    continue

```

3. Challenges of Using a Single Camera (and how to resolve them)

1. **No direct depth (range) from a single image.**

How I would address it: Use practical geometric cues such as depth-from-size when approximate drone dimensions are known, or keypoint-based pose estimation when sufficient structure is visible. If additional robustness is required, learned monocular depth models (e.g., lightweight networks such as FastDepth [4]) can provide a dense relative depth prior, while visual-inertial odometry or SLAM can supply scale-consistent motion cues. These methods would be treated as optional enhancements rather than core components, since the interception task operates in a limited workspace where simpler geometric constraints are often sufficient. Small controlled camera motions can further refine depth via parallax when needed.

2. **Motion blur and fast drone motion.**

How I would address it: Prefer higher FPS and short exposure. Use tracking + Bayesian filtering to smooth detections and recover through short dropouts. Keep the prediction horizon short so the system does not over-shoot when the drone changes direction.

3. **Occlusion during the final approach (gripper blocks the view).**

How I would address it: Slow down near capture and rely briefly on filter prediction when the drone is partially hidden. Close the gripper only when confidence is stable, and verify the grasp using gripper force feedback (and/or a simple visual confirmation after closure).

4. **Background clutter causing false positives.**

How I would address it: Train the detector with strong negative examples (birds, cars, small distant objects) and use temporal consistency: require detections to persist across multiple frames and follow plausible motion before acting on them.

5. **Calibration errors (camera mounting and transforms).**

How I would address it: Perform careful camera and hand-eye calibration, and include a quick validation routine during setup. Conservative safety margins can be used if residual calibration uncertainty is expected.

6. **Latency in the loop (camera + compute + robot actuation).**

How I would address it: Use short-horizon prediction from the filtered state to aim where the drone will be, not where it was seen. Keep the perception pipeline lightweight and run the control loop as fast as possible.

4. Alternative Approaches and Additional Sensors

To reduce depth ambiguity and improve robustness beyond a single monocular camera, I would consider the following options, grouped by practicality and deployment setting:

- **Stereo camera.**

Adds direct depth via disparity, improving 3D localization and interception timing without relying heavily on monocular assumptions.

- **Event camera + RGB fusion.**

Event cameras offer very low latency, high dynamic range, and strong robustness to motion blur, making them attractive for fast targets. Systems such as stereo fisheye event setups (e.g., SFERA [5]) demonstrate effective drone detection and tracking with wide field-of-view coverage. In practice, event data works best when fused with RGB semantics.

- **Radar (mmWave).**

Provides robust range and relative velocity even under poor lighting or partial occlusions. Radar-vision fusion is particularly useful for stabilizing tracking when the visual signal is unreliable.

References

References

- [1] H. Guo, X. Lin, and S. Zhao, “YOLOMG: Vision-based Drone-to-Drone Detection with Appearance and Pixel-Level Motion Fusion,” arXiv:2503.07115, 2025.
- [2] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” ECCV, 2022.
- [3] U. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” Ultralytics, 2023.
- [4] D. Wofk, F. Ma, T.-J. Yang, S. Karaman, and V. Sze, “FastDepth: Fast Monocular Depth Estimation on Embedded Systems,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [5] D. R. Da Costa, M. Robic, P. Vasseur, and F. Morbidi, “New Stereo Fisheye Event Camera for Fast Drone Detection and Tracking,” 2024.